Lecture 63

JWT (JSON Web Token)

JWT :- It's provides a secure way of transmitting information between parties as a JSON object.

- This information can be verified because its digitally signed using RSA (public/private key pair) etc.

Advantages :-

- Compact : Because of it's size, it can be sent inside an HTTP header itself. And due to its size its transmission is fast.

- Self Contained / stateless : The payload contains all the required information about the user, thus it avoid querying the database.

Concept && Coding

- Can be signed using Symmetric (HMAC) or Asymmetric (RSA).

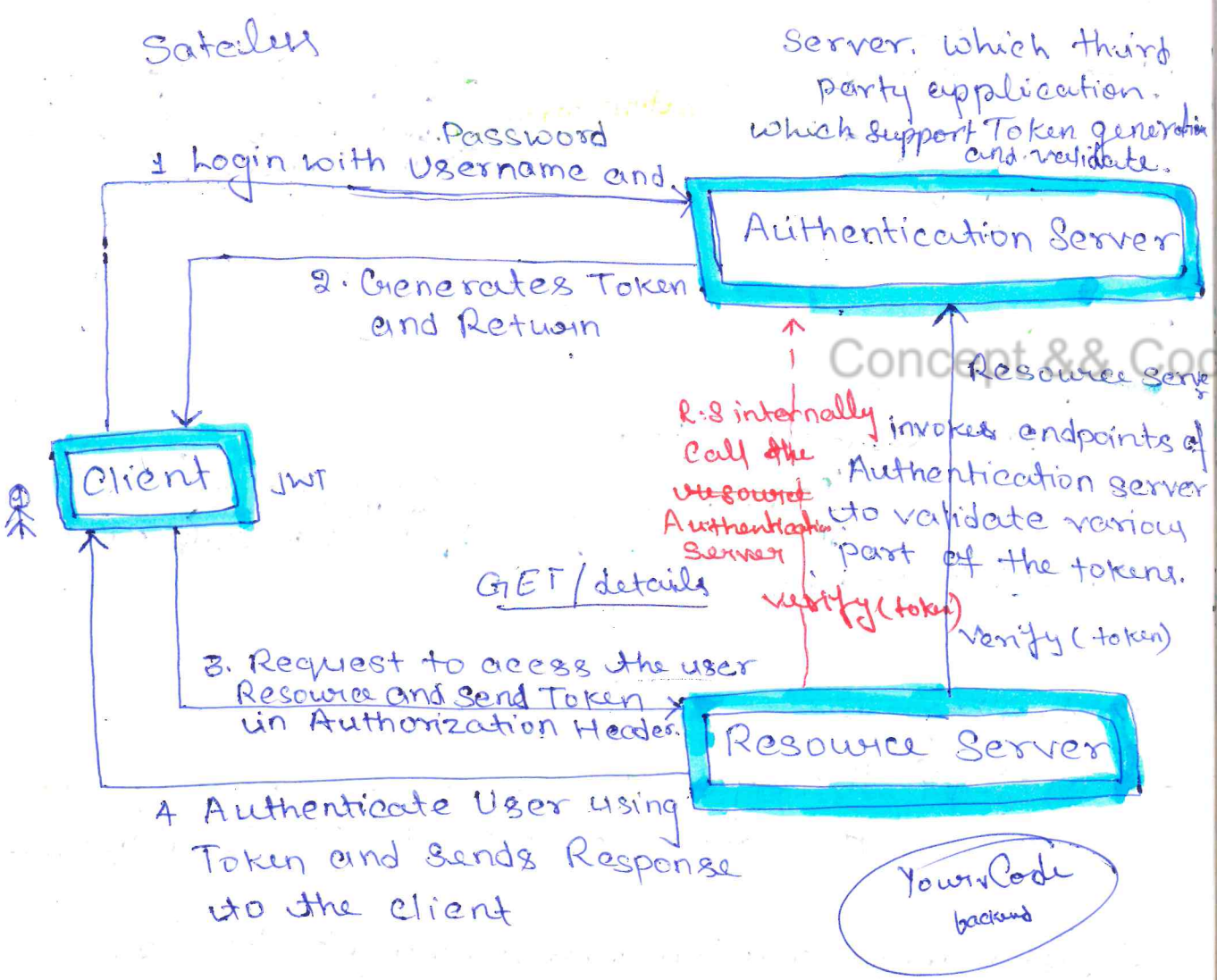
- Built in expiry mechanism.

- Custom claim (additional data) can be added in JWT.

Roles –

Where is JWT ?

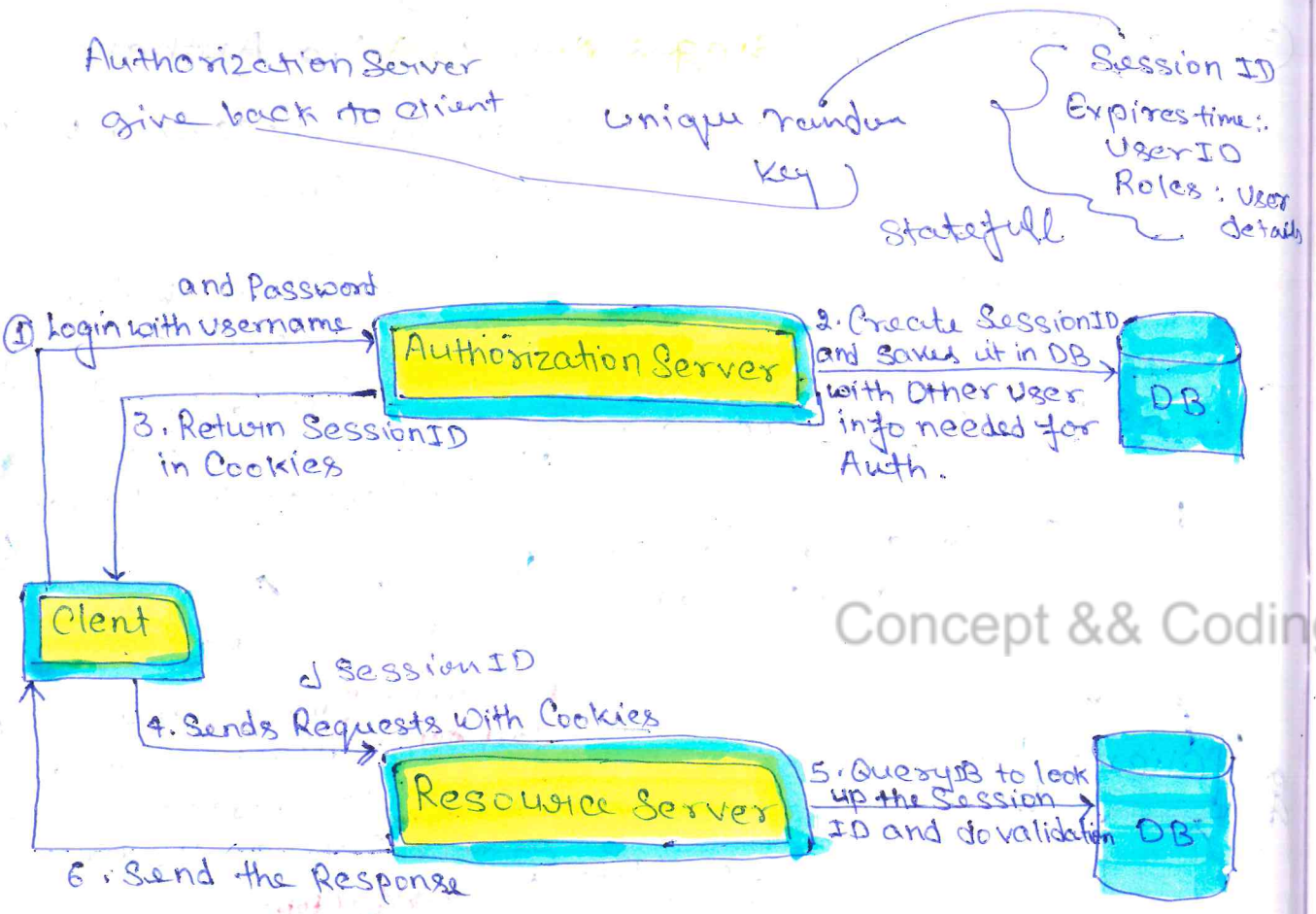
1. Used for **AUTHENTICATING** (Confirming the user identity)
2. Used for **AUTHORIZATION** (checks the user permission before providing access to resources)
3. Used for **SSO (Single Sign On)** i.e Authenticate once and access multiple application.



Concept && Coding

* Before we understant more about JWT, lets first understand, what was popular before JWT and what are the problems with it?

Session ID (or JSESSIONID)



Concept && Coding

Disadvantages:-

- **Stateful**: it rely on server side state management, it cause problem in distributed systems.

- It just a unique random, when server get this id, it has to perform DB query to fetch the details.

- Extra latency will happen to query on DB.

JWT Structure (JWT / JWS) JWE

JWT Structure

(aaaaaa . bbbbbbbbb . ccccccc)

1 → Header

2 → Payload

3 → Signature

```
{
  "type" : "JWT"
  "alg"  : "RSA"
  "kid"  : "sfdsf234234"
```

```
{
  "iss" : "https://example.com/auth"
  "sub" : "1234567890",
  "ex"  : 1711945909,
  "exmail" : "abc@gmail.com"
  "jti" : uniqi-id-12312312"
}
```

RSASHA256(
Base64URL(Header)
+ Base64URL(Payload)
)

Concept && Coding

Header :→

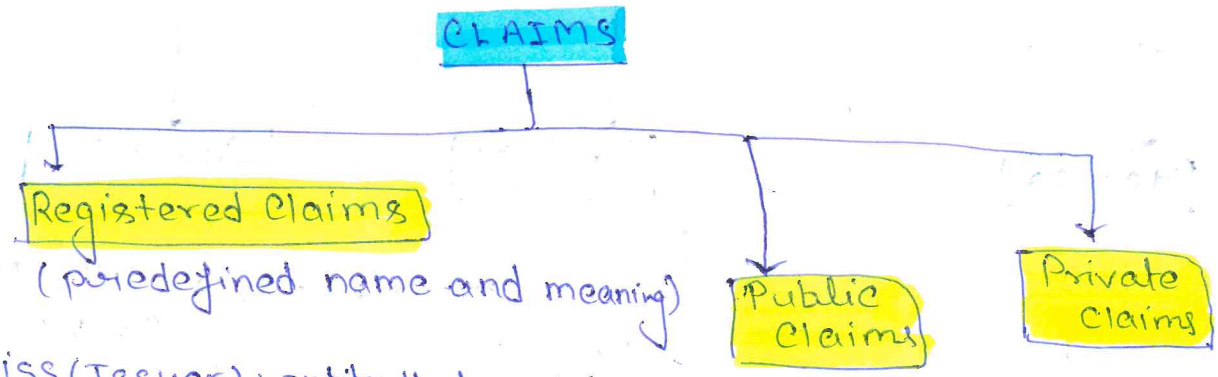
- Contains metadata information of the token
- type : Type of token, generally JWT always use add here.
- alg : Signing algorithm used like RSA or HMAC etc..

RSA → Asymetric Cryptographic key

HMAC → Symetric

Payload :-

- Contains claims (or in simple terms, User information or any additional information is kept here)

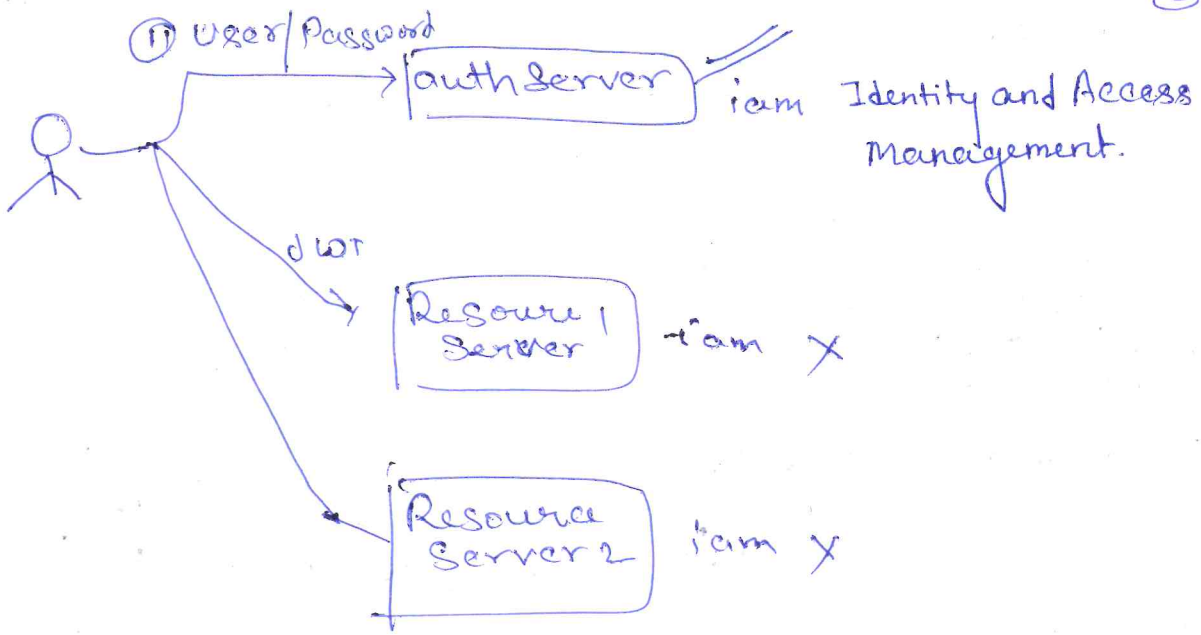


- iss (Issuer) : entity that ~~used~~ issued the JWT.
- sub (Subject) : Identifies the user.
- aud (Audience) : Identifies the recipient for which token is intended
- exp (Expiration Time): After this time, token is not valid
- nbf (Not Before): Time before this, Token should not be accepted
- iat (Issued At) : time at which token is issued.
- jti (JWT Id) : Unique JWT ID

Concept && Coding

Public claims :- It's a custom claim, whic can be shared and understood by multiple parties) - abc@gmail.com
role

Private claims :- It's a custom claim, which are intented for internal use only and not standardize, nor expected that other parties understand this)



Signature :- (JWT/JWS) — Json web Signature.

- Encode JWT Header and Payload separately using Base 64 encoding.
- Concatenate the Encoded Header and Payload string using ".".
  (ex: xxxxxx.yyyyyyy) This is known as Message
- Use RSA (Asymmetric cryptography) or HMAC (symmetric cryptography) to create digital signature.
- Encode the signature generated in previous step
- Concatenation using ".".

Concept && Coding

Header { Payload { Signature

RSA — unsymmetric key
HMAC — symmetric key

"Base 64 encode(Header)"

"base 64 encode(Payload)", Key

RSA → Private key
HMAC — Secret key

= base 64 (abcdefghij)

SK
Key1 Key1

RSA
Privat Public

Concept && Coding

* during signin it can use private key and verify we can use public key

"Base 64 encode(Header)".."base64encode(Payload)".
"Base 64 encode(Singuture)"

Sample API Request :

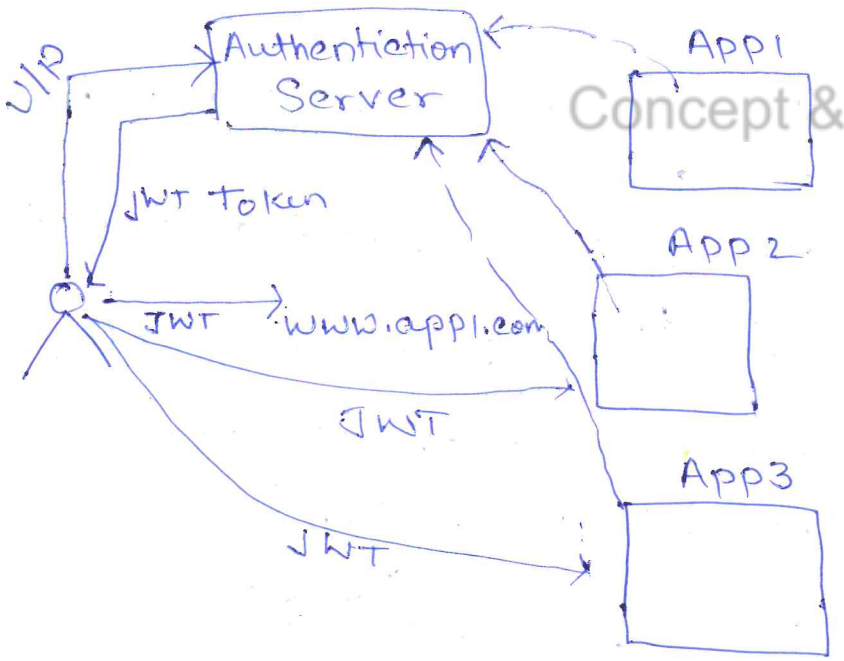
curl --location --request Get "https://localhost.89/api/resou
-header Authorization : Bearer

abcd . eacd . ~~cabcd~~

Challenges with JWT :-

1:- Token Invalidation : let say, I have blacklisted one user, how to invalidate its

SSO (Single Sign On)



Concept && Coding

~~Only~~ we have login Only Once place. we can SigIn with multiple app.

Challenges with JWT :→ — Stateless

1. Token invalidation : Let say, I have blacklist one user, how to invalidate its Token Before it's expiration ?

(a) Server need to keep the list of blacklisted tokens and then DB/cache lookup is required while validating.

(b) or, change the Secret key, but this will make the JWT invalidate for all users.

(c) or, Token ~~should~~ should be very short lived. *

(d) or, Token should be used only once. *

2. JWT token is encoded, not encrypted. So its ~~it~~ less secure.

(a) Use JWE, Means encrypt the payload part.

JSON Web Encrypt

3. Unsecured JWT with "alg" : none, such JWT should be rejected.

JWT / JWS

Header Payload

alg : none

Concept && Coding

*4:- JWK exploit:- public key shared in this, should not be used to verify the Signature.

e stands for "exponent" and n stands for "modulus" and combined together they from Public key.

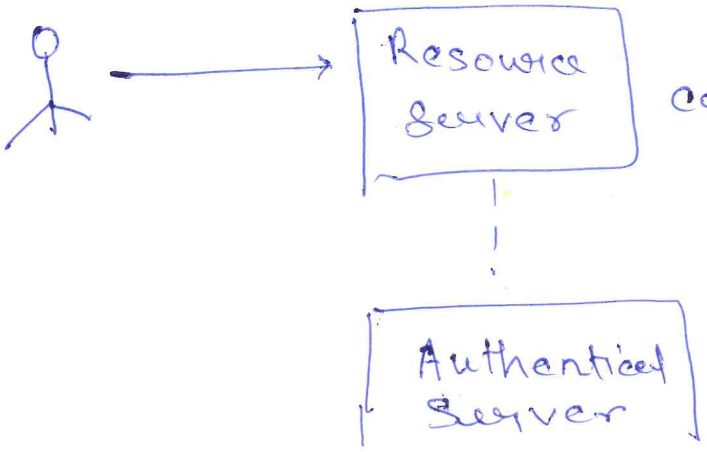
```
{
  "typ": "JWT",
  "alg": "RSA",
  "jwk": {
      "n": "sfsdf23432432 4fsd4S fdsfs df23",
      "e": "ABC04 ED"
      "kid": "sdfds343243243 2fwfdwfsfaf"
  }
}
```

Concept && Coding

Signature

Sigin (Message, Private Key)

Verify (Message, Public Key)



can not verify (Mess, Public

Solution:

5. Use "kid" in the Header to look up the https://{Auth Server domain}/.well-known/jwks.json to find the Public key

Concept && Coding